Table of Contents

1 Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

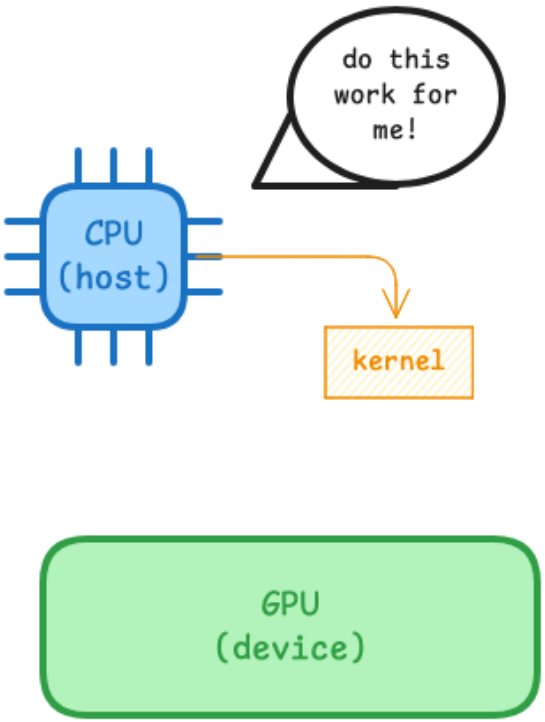
- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

Our goal is unchanged math with less data movement and more useful work per thread, and sometimes we need to accumulate matrix multiplications.

Example

During the forward pass when training a model, we accumulate the gradients from the entire batch.

CUDA Basics




```
int main() {
  const int M = 128, N = 128, K = 128;

  float *dA, *dB, *dC;
  cudaMalloc(&dA, M * K * sizeof(float);
  cudaMalloc(&dB, K * N * sizeof(float));
  cudaMalloc(&dC, M * N * sizeof(float));

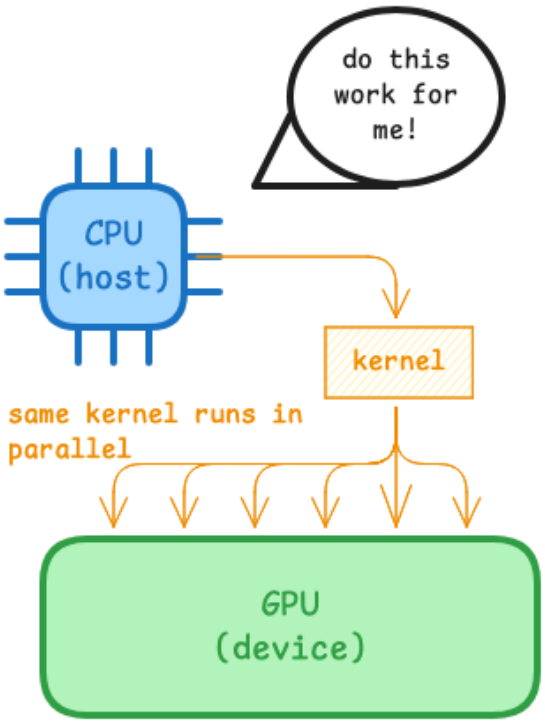
  // load values into these device pointers...

  run_sgemm_naive(M, N, K, 1, dA, dB, 0, dC);

  cudaFree(dA); cudaFree(dB); cudaFree(dC);
}
```

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.

CUDA Basics

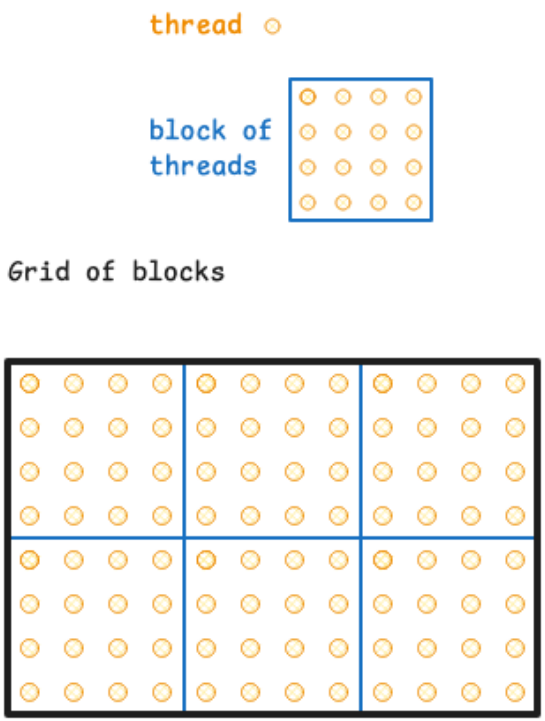



```
__global__ void sgemm_naive(
  int M, int N, int K, float alpha, const float *A,
  const float *B, float beta, float *C
) {
  // implement kernel ...
}

void run_sgemm_naive(int M, int N, int K, float alpha,
     float *A, float *B,
                        float beta, float *C) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```

- A kernel runs in parallel on many **threads** on the device.
- The number of threads is configured by the host, when it **launches** the kernel.

CUDA Basics




```
__global__ void sgemm_naive(
  int M, int N, int K, float alpha, const float *A,
  const float *B, float beta, float *C
) {
  // implement kernel ...
}

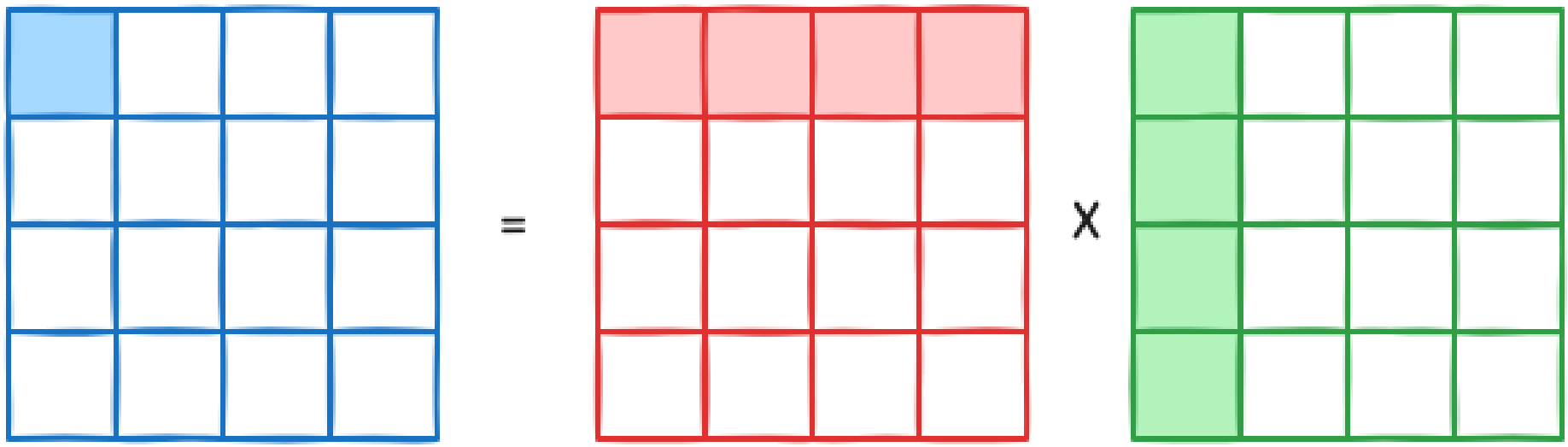
void run_sgemm_naive(int M, int N, int K, float alpha,
     float *A, float *B,
                       float beta, float *C) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```

- Threads are grouped into **blocks**.
- The collection of blocks requested by the host is organized in a **grid**.

One thread per output element

Recall

Recall that matrix multiplication is naturally parallel, and the output elements of C can be computed independently in parallel.

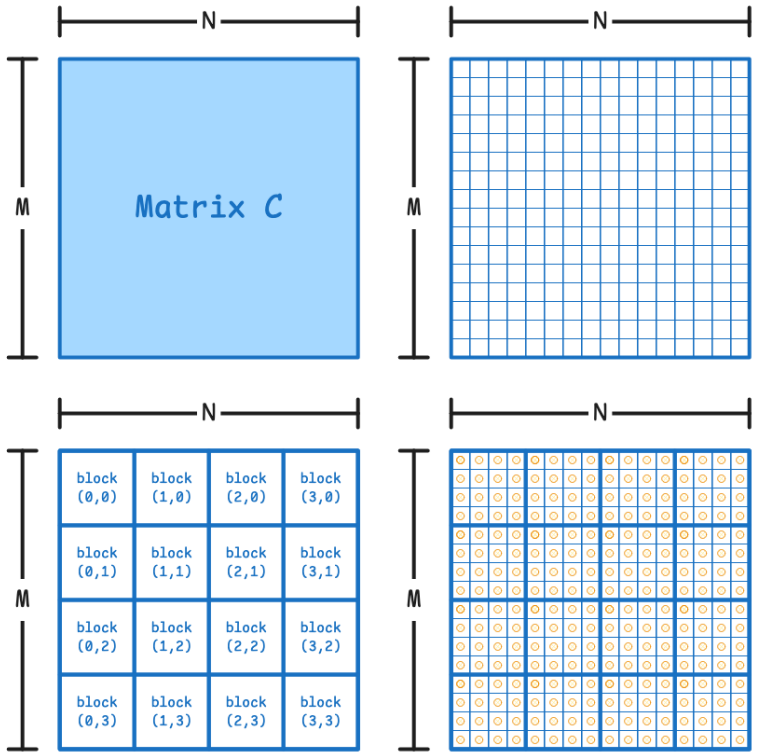


We can naturally map one thread to each output element!

Naive SGEMM Kernel

```
__global__ void sgemm_naive(
    int M, int N, int K, float alpha, const float *A,
    const float *B, float beta, float *C
) {
  const uint x = blockIdx.x * blockDim.x + threadIdx.x;
  const uint y = blockIdx.y * blockDim.y + threadIdx.y;

  if (x < M && y < N) {
    float tmp = 0.0;
    for (int i = 0; i < K; ++i)
      tmp += A[x*K+i] * B[i*N+y];
    C[x*N+y] = alpha*tmp + beta*C[x*N+y];
  }
}
void run_sgemm_naive(
  int M, int N, int K, float alpha, float *A,
  float *B, float beta, float *C
) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```



In Context

```
__global__ void sgemm_naive(int M, int N, int K, float alpha, const float *A,
                            const float *B, float beta, float *C) {
  const uint x = blockIdx.x * blockDim.x + threadIdx.x;
  const uint y = blockIdx.y * blockDim.y + threadIdx.y;

  if (x < M && y < N) {
    float tmp = 0.0;
    for (int i = 0; i < K; ++i)
      tmp += A[x*K+i] * B[i*N+y];
    C[x*N+y] = alpha*tmp + beta*C[x*N+y];
  }
}

void run_sgemm_naive(int M, int N, int K, float alpha, float *A, float *B,
                     float beta, float *C) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}

int main() {
  // ... set up matrices and dimensions ...
  run_sgemm_naive(M, N, K, alpha, A, B, beta, C);
}
```

Device
`__global__` marks a GPU **kernel**.

Many threads
Every thread runs this code with its own ID.

Host
CPU code manages the top-level logic.

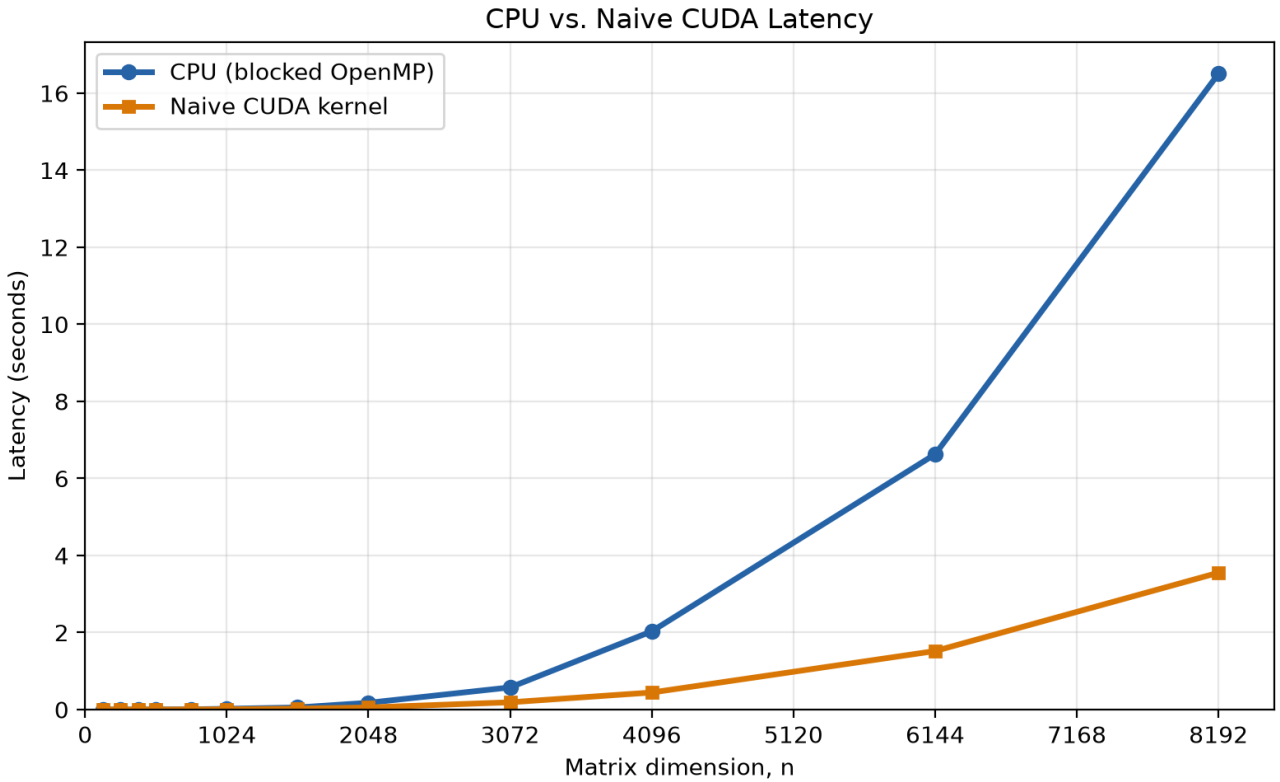
Launch
The host chooses the grid and block sizes.

Naive SGEMM visualization

- One thread computes one element of C.
- It walks one row of A and one column of B for the full dot product.
- Neighboring threads repeatedly fetch many of the same values from global memory.

Play naive-kernel visualization

Naive CUDA is already faster than the CPU



We can do better

These benchmarks were run on an RTX 3090 with specs:

- 10,496 CUDA cores over 82 SM's with up to 35.6 TFLOP/s of FP32 throughput.
- 24 GB of GDDR6X memory with 936 GB/s of bandwidth.

Theoretical lower bound on latency

For SGEMM with $n = 8192$:

$$\text{work} \approx 2n^3 = 1.10 \times 10^{12} \text{ FLOP},$$

$$t_{\min} \approx \frac{1.10 \text{ TFLOP}}{35.6 \text{ TFLOP/s}} \approx 0.031 \text{ s} = 31 \text{ ms}.$$

Observed latency: 3.55 s—about 115× the theoretical minimum.

What can we take advantage of?

1. **Accessing Device Memory is very slow.** 96.7% of warp stalls are memory-related.
2. Blocks are scheduled to SMs; threads are scheduled on CUDA cores within SMs.
3. Threads can save time by loading data from Device/Global memory into Shared Memory within the SM.

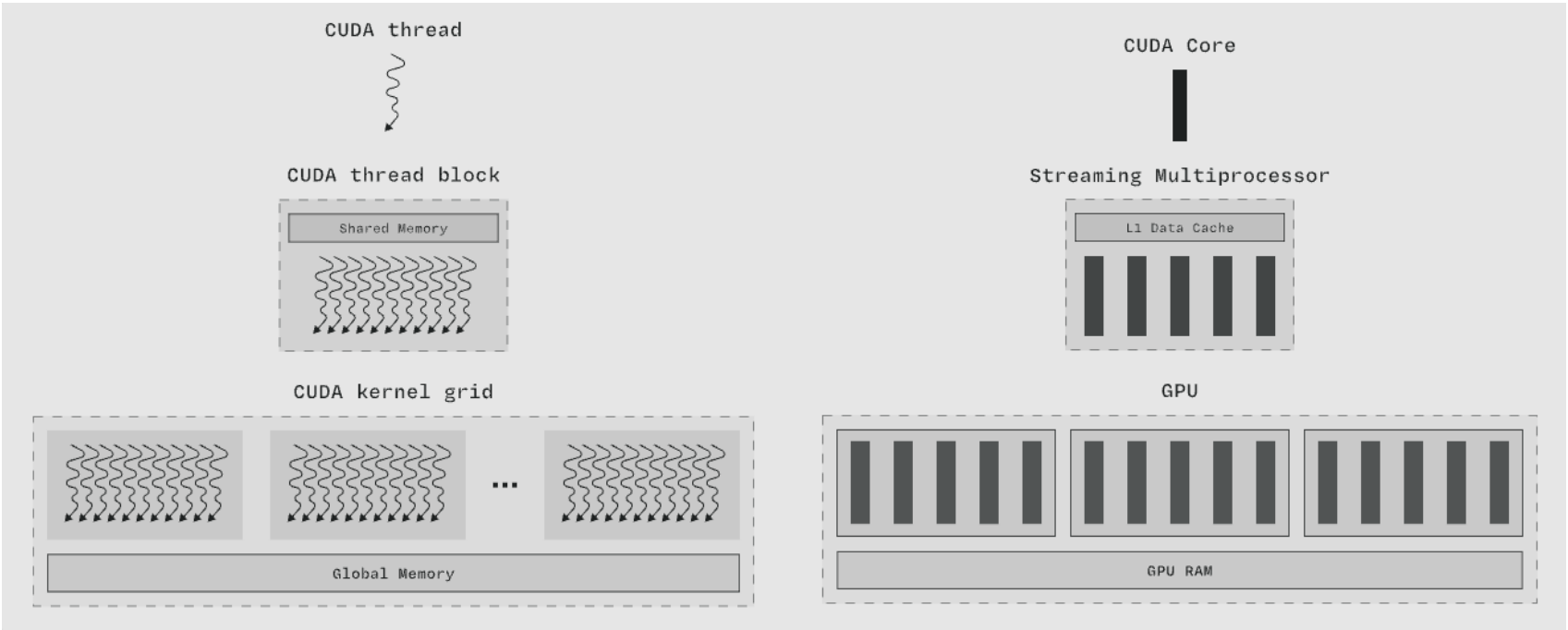
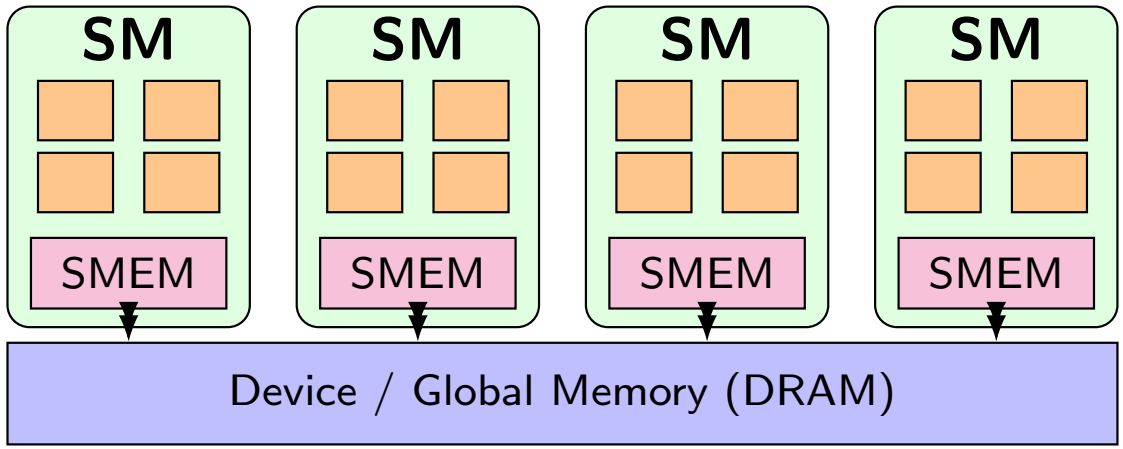


Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

Tiling

Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.



Basic tiled SGEMM: reuse data on chip

1. A block cooperatively loads a tile of A and a tile of B from global memory into shared memory.
2. Synchronize so every thread sees the complete tiles.
3. Each thread accumulates a partial dot product using the cached values.
4. Synchronize, advance along K, and repeat until the output tile is complete.

Basic tiled SGEMM

```
__shared__ float As[BLOCKSIZE * BLOCKSIZE];
__shared__ float Bs[BLOCKSIZE * BLOCKSIZE];
float tmp = 0.0;

for (int bkIdx = 0; bkIdx < K; bkIdx += BLOCKSIZE) {
  As[threadRow * BLOCKSIZE + threadCol] =
      A[threadRow * K + threadCol];
  Bs[threadRow * BLOCKSIZE + threadCol] =
      B[threadRow * N + threadCol];

  __syncthreads();
  A += BLOCKSIZE;
  B += BLOCKSIZE * N;

  for (int dotIdx = 0; dotIdx < BLOCKSIZE; ++dotIdx) {
    tmp += As[threadRow * BLOCKSIZE + dotIdx] *
           Bs[dotIdx * BLOCKSIZE + threadCol];
  }

  __syncthreads();
}
```

Cooperate
Each thread loads one value from A and one from B.

Synchronize
Wait until both shared tiles are complete.

Reuse
One thread accumulates from the cached row and column.

Advance
Protect the cache, move along K, and repeat.

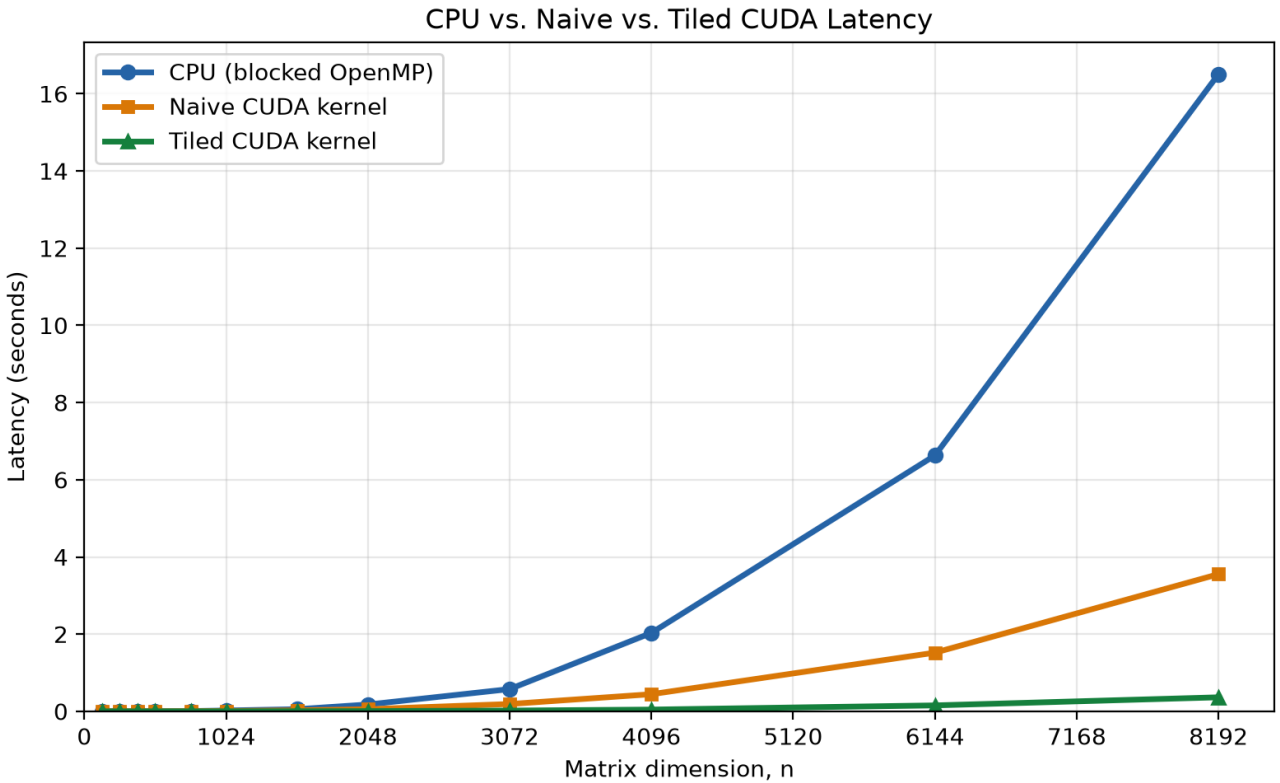
Adapted from Boehm (2022), kernel 3.

Basic tiled SGEMM visualization

- Watch each block load small regions of A and B together.
- The loaded tiles are reused to update an entire tile of C.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

Tiling cuts latency by another order of magnitude



Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms.

After profiling, we see that 76.4% of warp stalls are still memory-related. In other words, we are **memory bound**.

Memory bound means poor hardware utilization.

If a kernel is **memory bound** we can increase throughput by moving less data from DRAM. Since we have to read the entirety of both input matrices at least once in order to perform the computation, this must be done through **data reuse** via caching in SMEM. In general, we can't decrease the total FLOPs required for an operation.

Arithmetic intensity and the Roofline Model

Throughput

Formally stated, **throughput** is the total useful work (FLOPs) done per unit time. *We previously discussed theoretical peak throughput*.

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Example

Theoretical peak throughput of our 3090 is 35.6 TFLOP/s. We observed 360ms or 0.36s of latency for $n = 8192$, where the total FLOPs required is 1.10 TFLOPs.

Thus, we have an observed throughput of

$$\frac{1.10\text{TFLOPs}}{0.36\text{s}} \approx 3.05\text{TFLOP/s}.$$

Roughly 10% of peak throughput.

Arithmetic intensity and the Roofline Model

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Example

For $n = 8192$ and 32×32 tiles, the grid contains $256^2 = 65{,}536$ blocks. Each block takes 256 steps, loading one tile from both A and B per step, then writes one tile to C.

$$\text{bytes/block} = \left(256 \times 2 \times 32^2 + 32^2\right) \times 4 = 2{,}101{,}248 \text{ bytes},$$

$$\text{bytes total} = 65{,}536 \times 2{,}101{,}248 \approx \boxed{137.7 \text{ GB}}.$$

Arithmetic intensity and the Roofline Model

Example (cont.)

For $n = 8192$ and 32×32 tiles, the grid contains $256^2 = 65{,}536$ blocks. Each block takes 256 steps, loading one tile from both A and B per step, then writes one tile to C. Across the complete grid, the kernel moves

$$65{,}536 \left(256 \times 2 \times 32^2 + 32^2\right) \times 4 \approx 137.7 \text{ GB}.$$

With a total work count of 1.10 TFLOP, its arithmetic intensity is

$$\text{AI} = \frac{1.10 \times 10^{12} \text{ FLOP}}{137.7 \times 10^9 \text{ bytes}} \approx \boxed{8.0 \text{ FLOP/byte}}.$$

Arithmetic intensity and the Roofline Model

Roofline Model

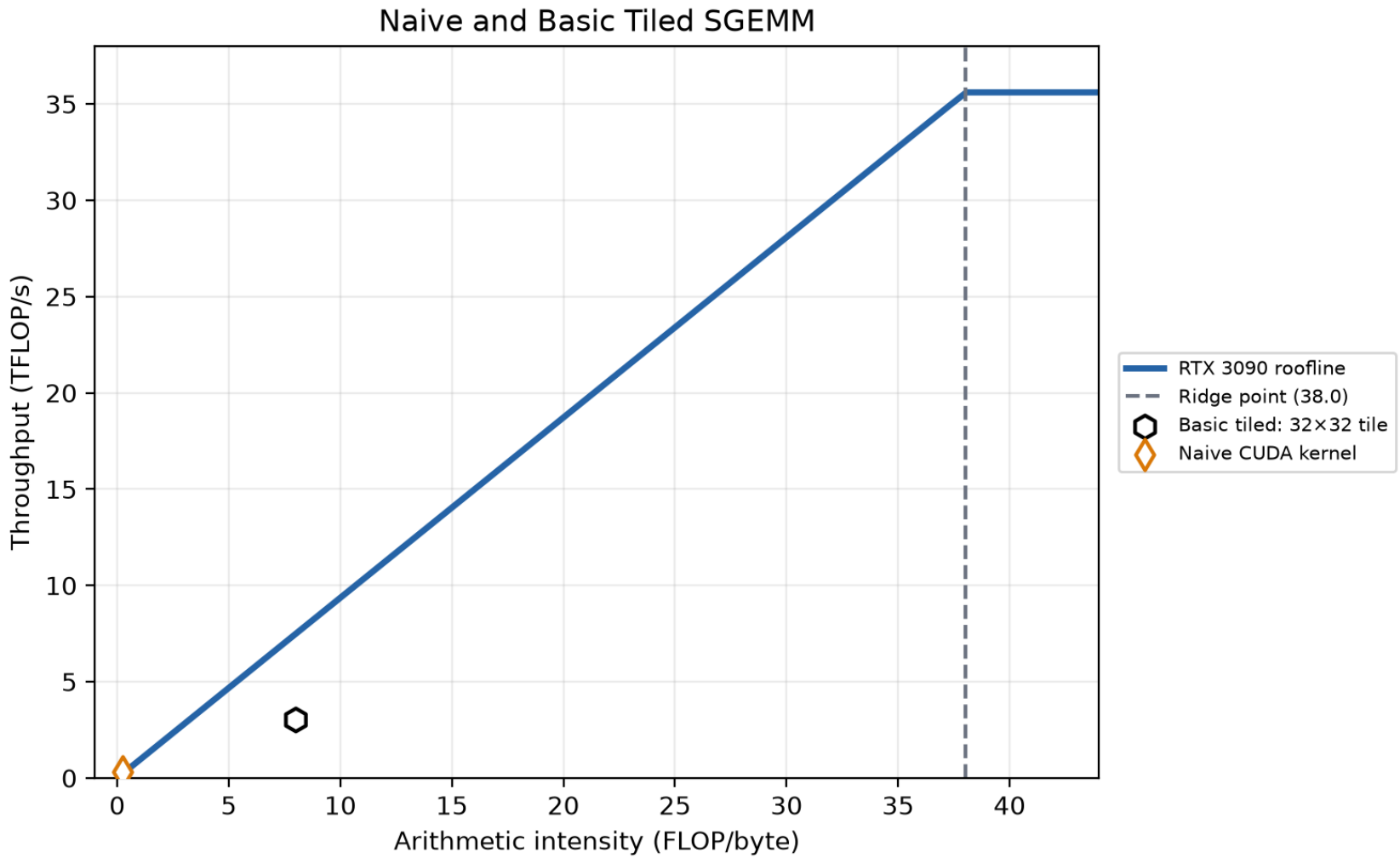
For a given system, we can predict a the upper bound of throughput of a kernel given its AI. Up to a certain point, increasing AI will raise the upper bound of throughput. However, at the **ridge point**, we become **compute bound**, and no longer benefit from increasing AI.

Example

For our 3090, we have 35.6 TFLOP/s of peak FP32 throughput and 936 GB/s of memory bandwidth. Thus, our ridge point is

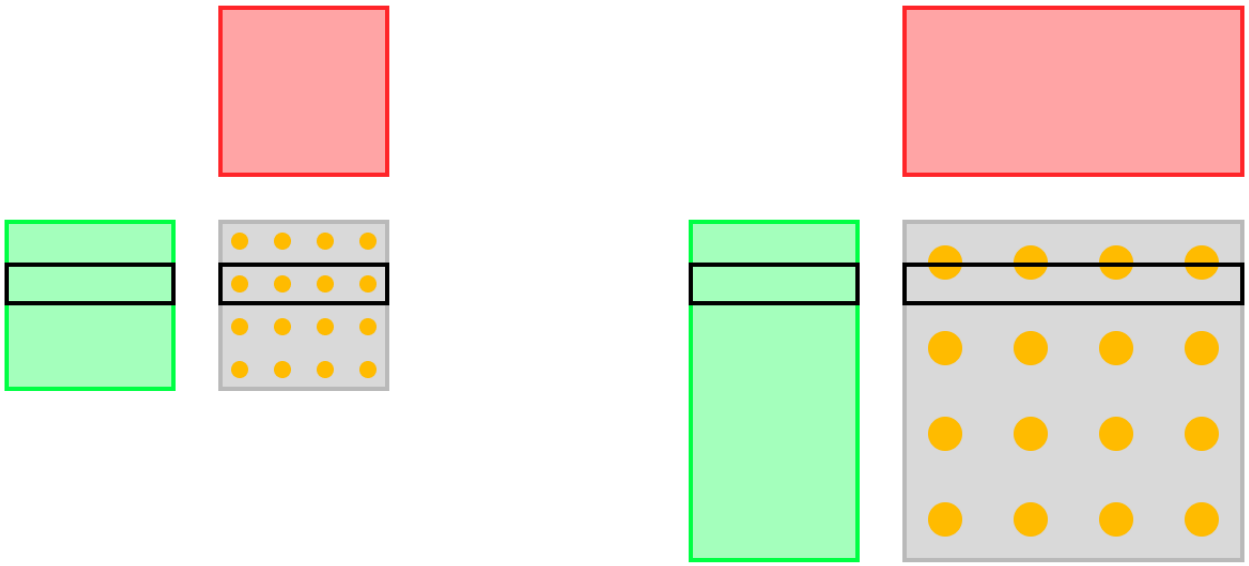
$$\text{RP} = \frac{35.6 \times 10^{12} \text{FLOP/s}}{936 \times 10^{9} \text{bytes/s}} \approx 38.0 \text{FLOPs/byte}$$

Arithmetic intensity and the Roofline Model



Thread coarsening: Increasing AI

- Basic tiling assigns 1 element of the output to each thread.
- We can *coarsen* each thread, and assign a $TM \times TN$ portion of the output to each thread.
- We load larger chunks into SMEM, increasing data reuse and AI.
- One loaded A value can update several columns; one loaded B value can update several rows.

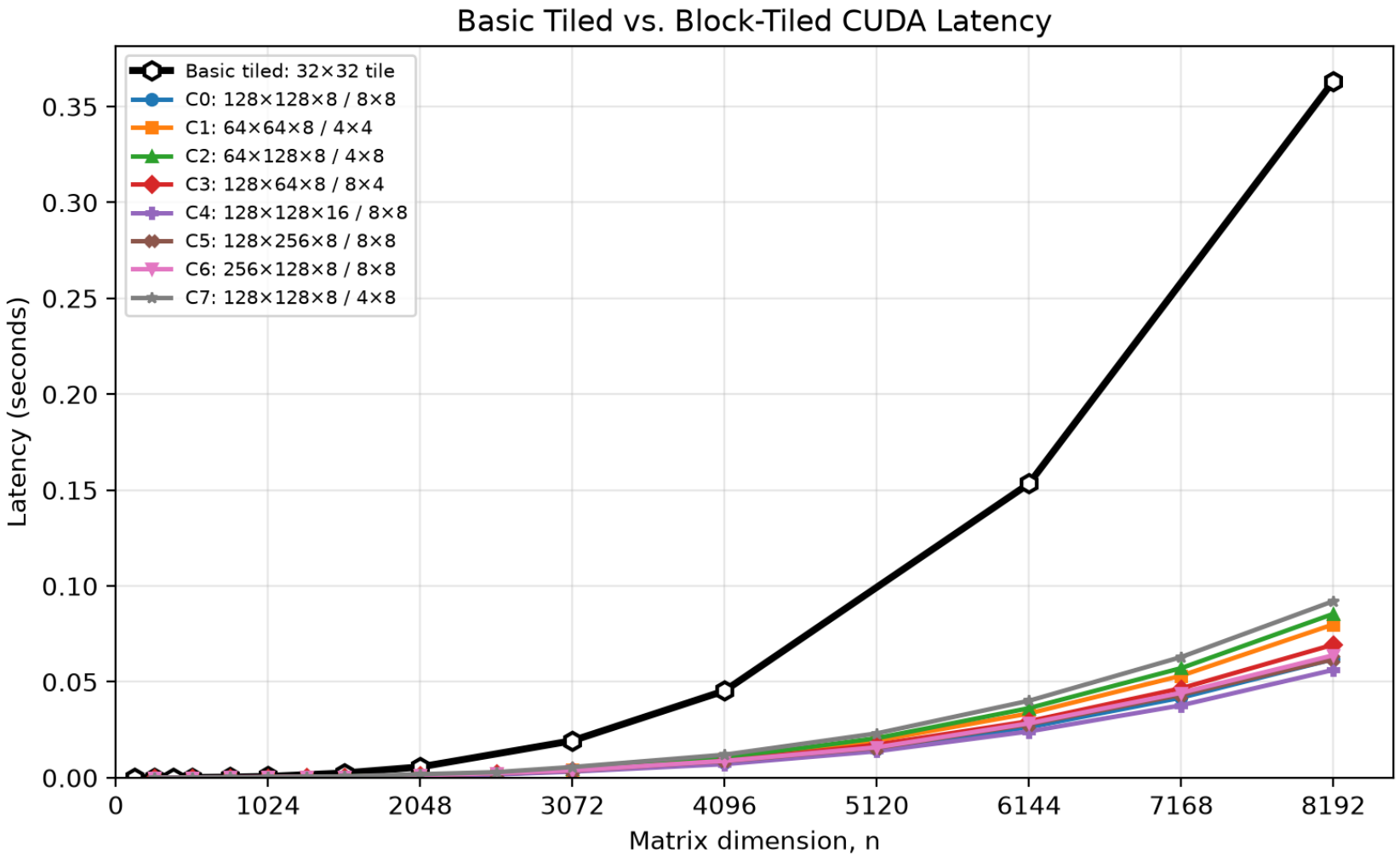


Coarsened SGEMM visualization

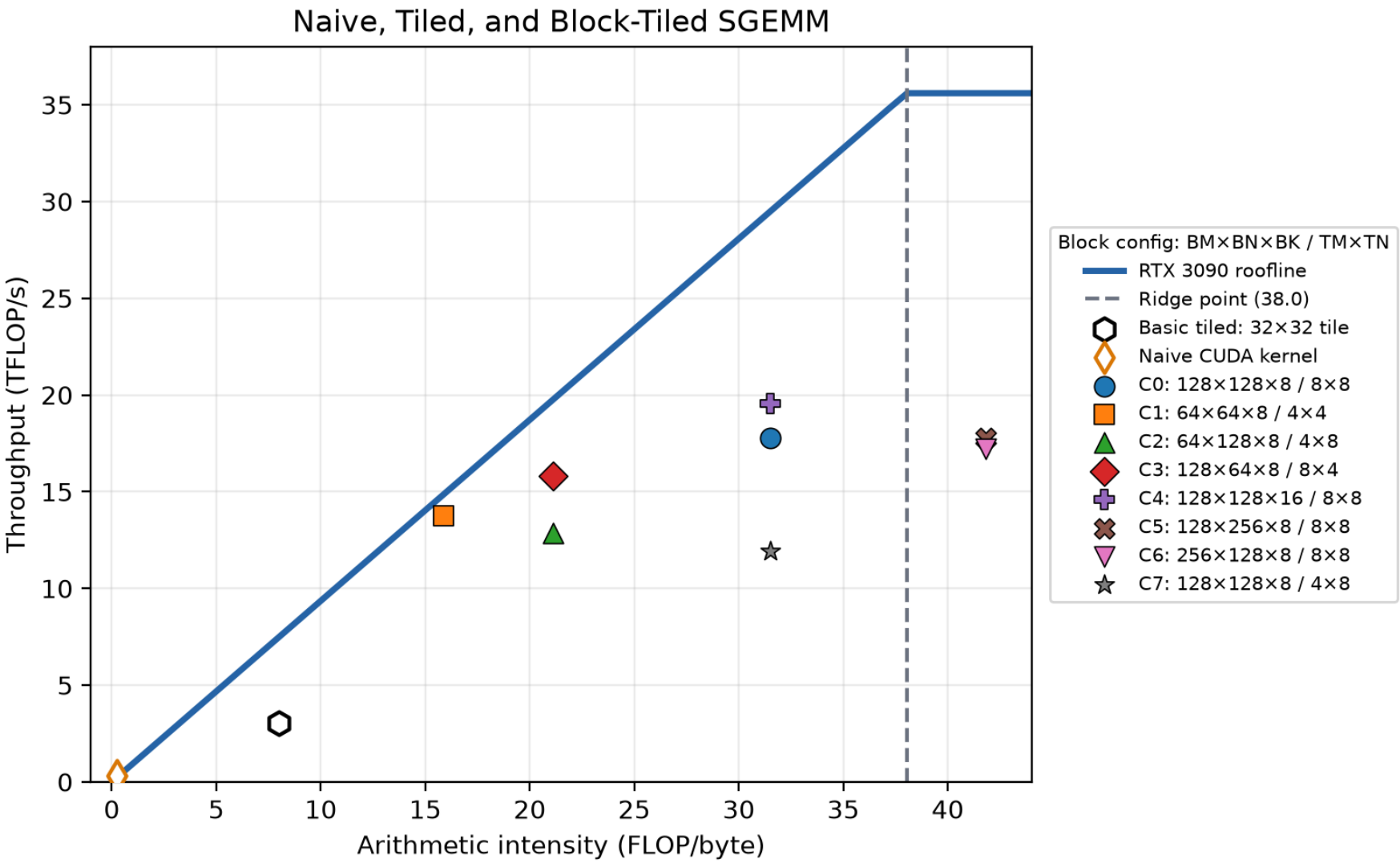
- Each thread now owns a small two-dimensional region of C.
- Values from shared memory are cached in registers and reused across that region.
- More outputs per thread means fewer memory operations per result.

Play coarsened-kernel visualization

Basic tiled vs. block-tiled latency



Tiling configurations on the roofline



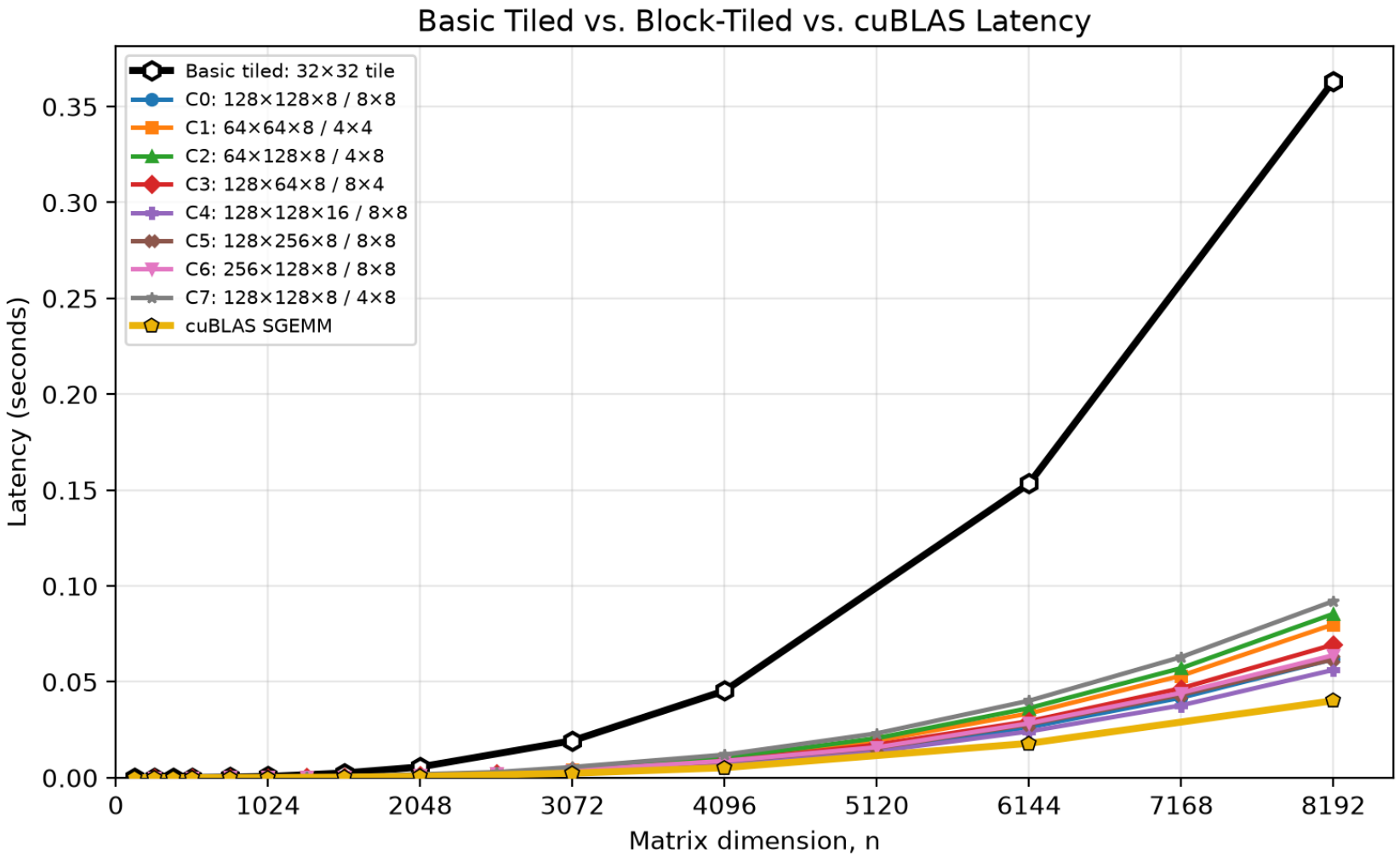
We have only scratched the surface: cuBLAS

- cuBLAS is NVIDIA's architecture-tuned linear algebra library and the practical performance target for our SGEMM kernels.
- At $n = 8192$, cuBLAS reaches 27.38 TFLOP/s, while our fastest configuration (C4) reaches 19.54 TFLOP/s, or 71.4% of cuBLAS's throughput.
- cuBLAS completes the multiplication in 40.2 ms versus 56.3 ms for C4, about 1.40× faster.

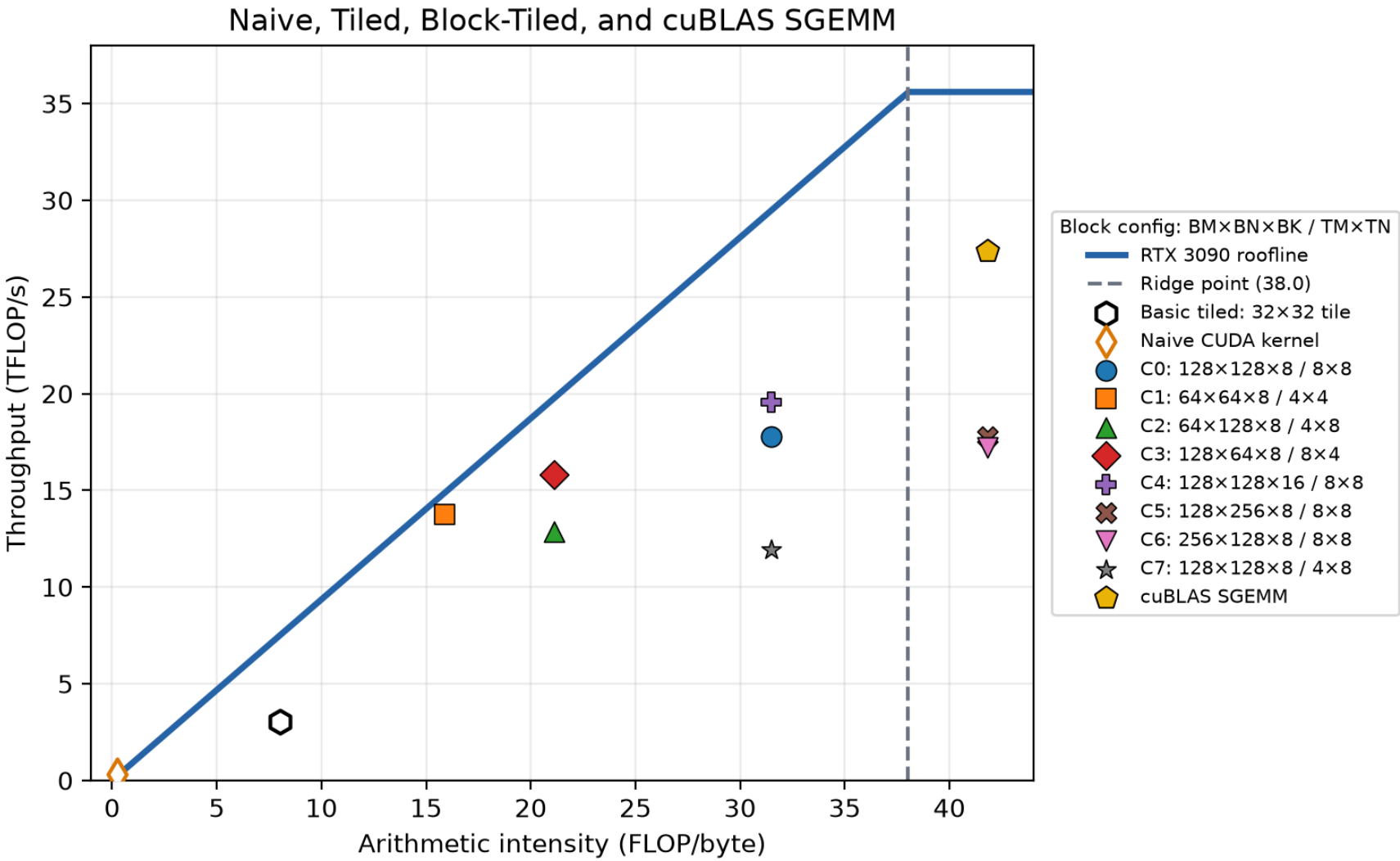
The remaining gap

Production kernels combine deeper register and warp tiling, vectorized memory access, pipelined loads, and architecture-specific instruction scheduling.

Basic tiled vs. block-tiled latency vs. cuBLAS



cuBLAS on the roofline



cuBLAS arithmetic intensity is estimated from the profiled 256×128 output-tile shape.

We've come a long way

